

Day 1 -Python Foundations

Assignment A -Employee Data Processing

1.

```
employee_data_processing.py X Extension: Python
employee_data_processing.py > ...
1  employee = [
2  {"id":1,"name":"John","salary":100000},
3  {"id":2,"name":"Jane","salary":120000},
4  {"id":3,"name":"Mike","salary":90000}
5  ]
6
7  def print_employees(data):
8      print("All employees")
9
10     for employee in data:
11         print (employee)
12
13     print_employees(employee)
14
15
```

```
PS C:\Users\DINITHI\Desktop\Day02_Assignment\AssignmentA> & C:\Users\DINITHI\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/DINITHI/Desktop/Day02_Assignment/AssignmentA/employee_data_processing.py
All employees
[{'id': 1, 'name': 'John', 'salary': 100000}, {'id': 2, 'name': 'Jane', 'salary': 120000}, {'id': 3, 'name': 'Mike', 'salary': 90000}]
[{'id': 1, 'name': 'John', 'salary': 100000}, {'id': 2, 'name': 'Jane', 'salary': 120000}, {'id': 3, 'name': 'Mike', 'salary': 90000}]
[{'id': 1, 'name': 'John', 'salary': 100000}, {'id': 2, 'name': 'Jane', 'salary': 120000}, {'id': 3, 'name': 'Mike', 'salary': 90000}]
```

2. Find the highest-paid employee.

```
17  # 2.Find the highest-paid employee.
18
19  def highest_paid(data):
20      highest = None
21      for employee in data:
22          salary = employee["salary"]
23          if salary is not None and salary >= 0:
24              if highest is None or salary > highest["salary"]:
25                  highest = employee
26      return highest
27  print("\n Highest paid employee:", highest_paid(employees))
28
29
30
```

```
Highest paid employee: {'id': 2, 'name': 'Jane', 'salary': 120000}  
PS C:\Users\DINITHI\Desktop\Day02_Assignment\AssignmentA> |
```

3. Calculate average salary.

```
# 3. Calculate average salary.  
  
def avg_salary(data):  
    total = 0  
    count = 0  
  
    for employee in data:  
        salary = employee["salary"]  
  
        if salary is not None:  
            total = total+salary  
            count = count + 1  
  
    if count == 0:  
        return None  
    return total / count  
  
print("\n Average salary:", avg_salary(employees))
```

```
Highest paid employee: {'id': 2, 'name': 'Jane', 'salary': 120000}  
  
Average salary: 103333.33333333333  
PS C:\Users\DINITHI\Desktop\Day02_Assignment\AssignmentA> |
```

4. Return employees earning more than 100,000.

```

45
46 # 4.Return employees earning more than 100,000.
47
48 def high_earners(data):
49     result = []
50
51     for employee in data:
52         salary = employee["salary"]
53
54         if salary is not None and salary > 100000:
55             result.append(employee)
56
57     return result
58
59 print("\n Employees earn more than 100000 are: ", high_earners(employees))
60
61

```

Average salary: 103333.33333333333

Employees earn more than 100000 are: [{ 'id': 2, 'name': 'Jane', 'salary': 120000}]

PS C:\Users\DINITHT\Desktop\Day02_Assignment\AssignmentA>

5. Count employees.

```

61 # 5.Count employees.
62 def count_employees(data):
63     return len(data)
64
65 print("\nNo of employees: ", count_employees(employees))

```

Employees earn more than 100000 are: [{ 'id': 2, 'name': 'Jane', 'salary': 120000}]

No of employees: 3

PS C:\Users\DINITHT\Desktop\Day02_Assignment\AssignmentA>

Assignment B -String & Validation

7. Implement `reverse_string("Hello")` → "olleH".

```
68 # 7.Implement reverse_string("Hello") → "olleH".
69
70 def reverse_text(text):
71     return text[::-1]
72
73 print("Reversed String: ", reverse_text("Hello"))
74
75
```

```
Reversed String: olleH
PS C:\Users\DINITHI\Desktop\Day02_Assignment\AssignmentA>
```

8. Implement `is_palindrome("madam")` → True

```
75 # 8.Implement is_palindrome("madam") → True
76
77 def is_palindrome(text):
78     return text == text[::-1]
79
80 print("\nIs madam palindrome - ", is_palindrome("madam"))
81
```

```
Is madam palindrome - True
PS C:\Users\DINITHI\Desktop\Day02_Assignment\Assignme
```

9. Validate that salary is numeric and non-negative.

```

82 # 9.Validate that salary is numeric and non-negative.
83
84 def valid_salary(salary):
85     if type(salary) not in (int, float):
86         return False
87
88     if salary < 0:
89         return False
90
91     return True
92
93 # 10.Handle invalid input without crashing.
94
95 print("-500 is valid: ", valid_salary(-500))
96 print("'hundred' is valid: ", valid_salary("hundred"))
97 print("1500 is valid: ", valid_salary(1500))
98

```

```

-500 is valid: False
'hundred' is valid: False
1500 is valid: True
PS C:\Users\DINITHI\Desktop\Day02_Assignment\AssignmentA>

```

. Day 1 Hard Scenario

```

99 # Day 1 Hard Scenario
100
101 Employees_with_missingSalary = employees + [
102     {"id": 4, "name": "Dini", "salary": None}
103 ]
104
105 print("Avg with missing salary: ", avg_salary(Employees_with_missingSalary))
106
107 # what should happen when the employee list is empty
108
109 print("\nAvg with no employees: ", avg_salary([]))

```

```

Avg with missing salary: 103333.33333333333
Avg with no employees: None
PS C:\Users\DINITHI\Desktop\Day02_Assignment\AssignmentA>

```